

Dear Reviewers,

In this attachment, we present a complementary experiment and some evidence to support our response to the constructive comments in the rebuttal.

1. FN Analysis in Large-Scale Dataset for Reviewer#A-Q4

Table 1: False Negatives of SSR in Large-Scale Dataset

Defect Type	# FN	FN rate (%)
Staking Logical Variables Manipulation (SVM)	1	1.04
Rewards without Timedelay (RT)	2	2.08
Single Liquidity Pool Reliance (SLR)	2	2.08
Omission in Status Update (OSU)	3	3.13
Unsafe Verification (UV)	2	2.08
Unauthorized Staking Asset Access (UAA)	4	4.17
Total	11	11.46

In addition, to validate the presence of logical defects that were not identified by SSR, we performed random sampling at a 95% confidence level with a 10% confidence interval on contracts for which SSR produced negative detection results. We randomly sampled and manually verified 96 such contracts in which no logical defects were detected by SSR. The experimental results are presented in Table 1. These results indicate that SSR exhibits a false negative rate of 11.46% on the large-scale dataset.

2. Details of FP and FN Analysis for Reviewer#A-W3, Reviewer#W2

False Positives. For Rewards without Timedelay (RT), Omission in Status Update (OSU), and Unsafe Verification (UV), the primary causes of false positives arise from limitations in the model’s ability to capture complex staking logic during the DeFi staking modeling process.

```
1 mapping (address => uint) public stakingTime; //used
  for the unstaking locktime
2 mapping (address => uint) public firstTime; //used
  for the APY boost
3 mapping (address => uint) public lastClaimedTime; //
  used for the claiming locktime
4 mapping (address => uint) public progressiveTime; //
  used for the claiming locktime
```

Figure 1: An example of Unauthorized Staking Asset Access defect

For example, as illustrated in Figure 1, the contract defines several variables related to users’ staking times. Although these variables share the same data structure, their semantics and functionalities differ. Specifically, the variable *stakingTime* records the time threshold at which a user can withdraw their staked assets; *firstTime* logs the timestamp of a user’s initial staking activity, which is used to calculate bonuses for the annual percentage yield (APY); *lastClaimedTime* captures the most recent time staking rewards were claimed, serving to calculate the staking reward; and *progressiveTime* is used to compute progressive staking rewards.

SSR struggles to accurately identify the complex logic related to staking time in DeFi staking modeling. As a result, it fails to determine which staking time variables the staking rewards depend

on. Additionally, it cannot distinguish which time variables need to be updated following a staking reward claim and which do not. In this case, *lastClaimedTime* needs to be updated following a staking reward claim, while *firstTime* does not. However, SSR incorrectly assumes that *firstTime* must also be updated, leading to a false OSM defect report.

For Staking Logical Variables Manipulation (SVM), Single Liquidity Pool Reliance (SLR), and Unauthorized Staking Asset Access (UAA), the primary cause of false positives arises from inaccuracies in pattern-based permission analysis and variable identification. For instance, due to incorrect recognition of permission controls, SSR mistakenly identifies certain functions, such as those intended for adjusting project strategies like modifying the transaction fee rate, which should be restricted to project developers, as callable by any external account. Since staking rewards are influenced by the transaction fee rate, this misidentification leads to a false SVM defect report.

False Negatives. For RT, OSU, and UV, the primary cause of false negatives lies in the limitations of the LLM, which lead to the incorrect identification of DeFi staking-related variables. For instance, if the LLM mistakenly identifies a variable unrelated to stake time as the stake time, it could result in the omission of an RT defect.

For SLR, SVM, and UAA, the primary cause of false negatives stems from errors or omissions in the control flow graph (CFG) and data flow graph (DFG) generated by Slither. For example, inaccuracies in the DFG may hinder the identification of all variables involved in the staking reward calculation. This could result in missing a variable that may be subject to external manipulation, leading to the omission of an SVM defect.

3. Construction of CDG for Reviewer#A-W1

Algorithm 1 Construction of CDG

Require: Transferred amount variable Var_{amount}
Require: Smart Contract $contract$
Ensure: Calculation Dependency Graph CDG

```

1:  $CDG \leftarrow \emptyset$ 
2:  $queue \leftarrow [Var_{amount}]$ 
3:  $visited \leftarrow visited \cup \{Var_{amount}\}$ 
4: while  $queue$  is not empty do
5:    $Var_{cur} \leftarrow queue.dequeue()$ 
6:    $Dep_{cal} \leftarrow GetCalculationDepend(Var_{cur}, contract)$ 
7:   for each  $Var_{cal}$  in  $Dep_{cal}$  do
8:     if  $Var_{cal} \notin visited$  then
9:        $queue.enqueue(Var_{cal})$ 
10:       $visited \leftarrow visited \cup \{Var_{cal}\}$ 
11:     end if
12:      $CDG.addCalculationEdge(Var_{cur}, Var_{cal})$ 
13:   end for
14:    $Dep_{con} \leftarrow GetControlDepend(Var_{cur}, contract)$ 
15:   for each  $Var_{con}$  in  $Dep_{con}$  do
16:     if  $Var_{con} \notin visited$  then
17:        $queue.enqueue(Var_{con})$ 
18:        $visited \leftarrow visited \cup \{Var_{con}\}$ 
19:     end if
20:      $CDG.addCalculationEdge(Var_{cur}, Var_{con})$ 
21:   end for
22: end while

```

Algorithm 1: Construction of Calculation Dependency Graph (CDG)

The algorithm for constructing Calculation Dependency Graph (CDG) is presented in

Algorithm 1. Starting with a variable that represents the transferred token amount, the algorithm extracts all other variables on which it depends. This dependency analysis is recursively applied to the newly identified variables until no additional dependencies are found. The CDG contains two types of edges, computation dependencies and control dependencies, which are determined by analyzing the variables read by the nodes in the DFG and CFG associated with the given variable. Once the CDG is constructed, state variables dependent on the transferred token amount can be identified. Variables that do not contribute to the detection of logical defects, such as local variables and input parameters, are removed to improve analysis precision.

```

1 function getPerTokenReward() public view returns (
2     uint) {
3     if ( LPInstance.balanceOf(address(this)) == 0) {
4         return 0;
5     }
6     uint newPerTokenReward = (totalStakeReward -
7         lastTotalStakeReward) * 1e18 / LPInstance.
8         balanceOf(address(this));
9     return PerTokenRewardLast + newPerTokenReward;
10 }
11
12 function pendingToken(address account) public view
13     returns(uint) {
14     unit _rewardPerToken = getPerTokenReward()
15     unit rewardAmount = userLPStakeAmount[account]
16         * (_rewardPerToken - userRewardPerTokenPaid[
17             account])
18         / (1e18)
19         + (userRewards[account]);
20     return rewardAmount
21 }
22
23 function getReward() public updateReward(msg.sender)
24 {
25     uint _reward = pendingToken(_msgSender());
26     require(_reward > min, "The stake reward is less
27         than the minimum obtainable reward.");
28     userRewards[_msgSender()] = 0;
29     if (_reward > 0) {
30         _standardTransfer(address(this), _msgSender
31             (), _reward);
32         return ;
33     }
34 }

```

Figure 2: Concrete example for constructing CDG

Consider the contract in the Figure 2 as an example. When a user invokes the *getReward()* function to claim staking rewards, the contract transfers a token amount, denoted as *_reward*, to the user (line 23). The process of constructing the Control and Computation Dependence Graph (CDG) for the *_reward* variable, which represents the staking reward transfer amount, is as follows:

(1) CDG Initialization: Begin with a CDG that only contains the *_reward* variable.

(2) Analyzing Dependencies of *_reward*: This includes both computation and control dependencies:

- Computation Dependencies: The value of variable *_reward* is calculated by invoking the *pendingToken()* function (line 19). Analysis of this function reveals that *_reward* depends on the variables *_rewardPerToken*, *userLPStakeAmount*, *userRewardPerTokenPaid*, and *userReward* (lines 10–11). These variables are connected to *_reward* in the CDG through computation dependency edges.

- Control Dependencies: The reward transfer operation (line 23) is conditional on a preceding check to verify whether the reward amount exceeds the minimum obtainable reward, *min* (line 20). Therefore, the variable *min* is linked to *_reward* via a control dependency edge.

(3) Recursive Expansion: For each newly identified and unanalyzed variable node, repeat step (2) to discover and connect its dependent variables until all variable nodes in the CDG have been analyzed. For instance, the variable *_reward* depends on *_rewardPerToken* (line 10), which, in turn, depends on *totalStakeReward*, *lastTotalStakeReward*, *LPInstance.balanceOf(address(this))*, and *PerTokenRewardLast* (lines 5–6). These dependencies are represented in the CDG through computation dependency edges. Furthermore, the computation of *_rewardPerToken* itself depends on the execution of *LPInstance.balanceOf(address(this))* (lines 2–4), introducing a control dependency edge from this variable to *_rewardPerToken*.

4. Supplementary Explanation of Technical Details for Reviewer#A-W1

Table 2: Semantics Related to Logical Defects in DeFi Staking

Name	Description
<i>lpPool(con)</i>	Variable <i>con</i> denotes an external liquidity pool contract.
<i>ModifyVar(func, var)</i>	The value of the variable <i>var</i> is modified within the function <i>func</i> .
<i>VerifyVar(func, var)</i>	The execution of certain logic within the function <i>func</i> requires the verification of variable <i>var</i> .
<i>NaTokenTrans(func, to)</i>	In function <i>func</i> , the native token of the contract is transferred to the address <i>to</i> .
<i>RewardTrans(func, from, to)</i>	In function <i>func</i> , the staking reward of the address <i>from</i> is transferred to the address <i>to</i> .
<i>permissCheck(func, addr)</i>	The execution of certain logic within the function requires verifying whether the function caller is the address <i>addr</i> or possesses authorization for <i>addr</i> .

SSR analyzes the semantics of liquidity pool contracts by examining their features, including the presence of functions such as mint and burn, as well as the ability to determine the ratio between two distinct tokens within the pool. These features are inferred through the analysis of widely adopted third-party liquidity pool contracts, including Uniswap and PancakeSwap.

SSR analyzes the semantics of functions by examining their control flow graph (CFG) and data flow graph (DFG), along with their specific features.

ModifyVar and *VerifyVar* indicate whether a function modifies or verifies the value of a specific variable, respectively. For *VerifyVar*, SSR examines certain special nodes in a function’s CFG, such as modifiers and require statements, to determine whether these nodes read a particular variable. If so, it indicates that the logic controlled by these special nodes depends on verifying the value of the variable. For *ModifyVar*, SSR checks whether any function in the contract performs a write operation on a specific state variable. If so, it indicates that the function is capable of modifying the value of the variable. By modeling the two-layer data structure of variables in the DeFi Staking Modeling module, SSR can identify functions that modify or verify these variables, as well as the special keys that are modified or verified.

NaTokenTrans and *RewardTrans* represent the transfer operations of native tokens and staking rewards within the functions, respectively. SSR identifies token transfer operations based on the

naming and parameter conventions defined in the ERC standards, and then determines whether the transferred token is a native token or a reward token. Specifically, it recognizes native token transfers by detecting the special variable *this.balance* and identifies staking reward transfers using the DeFi staking model.

5. Identification of Access Control Mechanisms for Reviewer#B-Q1

Taking the two common permission control mechanisms, modifiers (e.g. *onlyOwner*) and require statements, as examples. SSR first extracts the control-flow nodes that govern a function's execution from the contract's control flow graph (CFG). Based on this, it then identifies the modifier functions and require statements invoked by the function.

It then analyzes whether these nodes contain logic that verifies whether the special variable *msg.sender* corresponds to a specific address. If such logic is present, SSR considers it a permission check performed within the function.

6. Example of DeFi Staking Model Evaluation for Reviewer#A-Q5

1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4
5	5	5	5
6	6	6	6
7	7	7	7
8	8	8	8
9	9	9	9
10	10	10	10
11	11	11	11
12	12	12	12
13	13	13	13
14	14	14	14
15	15	15	15
16	16	16	16
17	17	17	17
18	18	18	18
19	19	19	19
20	20	20	20
21	21	21	21
22	22	22	22
23	23	23	23
24	24	24	24
25	25	25	25
26	26	26	26
27	27	27	27
28	28	28	28
29	29	29	29
30	30	30	30
31	31	31	31
32	32	32	32
33	33	33	33
34	34	34	34
35	35	35	35
36	36	36	36
37	37	37	37
38	38	38	38
39	39	39	39
40	40	40	40
41	41	41	41
42	42	42	42
43	43	43	43
44	44	44	44
45	45	45	45
46	46	46	46
47	47	47	47
48	48	48	48
49	49	49	49
50	50	50	50
51	51	51	51
52	52	52	52
53	53	53	53
54	54	54	54
55	55	55	55
56	56	56	56

Figure 3: Example of DeFi Staking Model Evaluation

Figure 3.a illustrates an example of a DeFi Staking Model constructed by SSR, while Figure 3.b shows the corresponding ground truth.

$$Acc_{model} = \frac{Acc_{var} + Acc_{func} + Acc_{cal}}{3}$$

The DeFi Staking Model comprises three components: Variables, Functions, and Calculations. The overall model accuracy is defined as the average accuracy of these three components. (In this context, accuracy metrics may include common evaluation indicators such as Precision, Recall, or F1-Score, and this applies similarly throughout the document.)

The methods for calculating the accuracy of the Variables, Functions, and Calculations components are as follows:

The Variables component consists of five types of variables with different semantics. The overall accuracy for the Variables component is calculated as the average accuracy across these five variable types:

$$Acc_{var} = \frac{Acc_{StakeAmount} + Acc_{StakeReward} + Acc_{StakeTime} + Acc_{StakeAddress} + Acc_{RewardAddress}}{5}$$

The accuracy for each variable type is calculated by comparing the corresponding variable lists extracted from the SSR-constructed model and the ground truth.

The accuracy for the Functions and Calculations components is computed similarly to the Variables component: by averaging the recognition accuracies of their respective subtypes.

7. Modification to the Usage of Formal Notation for Reviewer#A-W2

7.1 Removal of Unnecessary Formal Notations

We have removed some unnecessary formal notations in Section 4-D of paper and revised the corresponding descriptions to make them clearer. The modified content is as follows:

“The structure of the DeFi staking model consists of three components: DeFi staking-related variables, functions, and calculations of the transferred token amounts. Specifically, the variables include the User Stake Amount, User Stake Reward, User Stake Duration, Stake Token Address, and Reward Token Address. The functions encompass three core operations in DeFi staking: staking, claiming rewards, and unstaking. The calculations of the transferred token amounts involve determining the number of equity tokens acquired through staking, the stake rewards earned, and the number of tokens received upon unstaking.”

“For the target transfer amount variable, the nodes in the CDG represent the variables on which the transfer amount depends, while the edges represent the dependencies between these variables. There are two types of directed edges in the CDG: Ecal and Econ, which denote the dependency between variable values in computations and the control flow relationship of variable

modification operations, respectively. If a variable, *varA*, is connected to *varB* by an *Ecal* edge, it indicates that the value of *varA* is calculated based on *varB*. If the edge is of type *Econ*, it indicates that the statement modifying the value of *varA* is controlled by *varB*.”

7.2 Modification of the Conventions for Certain Formal Notations

We have modified the formulas in Section IV-F of paper based on the comments to make them more compliant with the standards. The specific modified content is as follows:

“(1) Staking Logical Variables Manipulation (SVM) SSR identifies two scenarios that may lead to this defect: dependency on ordinary variables and dependency on the balance of the native token. The detection logic for these scenarios is specified in Rule 1 and Rule 2, respectively. Rule 1 asserts that a defect exists if the staking reward (*Reward(re)*) is calculated based on a variable (*CalDepend(re, var)*), and this variable’s value can be altered by an external function. Rule 2 states that a defect is present if the staking reward is calculated based on the balance of native token (*DependonBalance(re)*), and the native tokens can be transferred by an external function (*NaTokenTrans(func,)*). In addition to the scenario involving staking rewards, SSR applies similar logic to detect cases related to tokens deposited during staking and unstaking.

$$isSVM = Reward(re) \wedge CalDepend(re, var) \wedge ModifyVar(func, var) \quad (1)$$

$$isSVM = Reward(re) \wedge DependonBalance(re) \wedge NaTokenTrans(func, _) \quad (2)$$

(2) Rewards without Timedelay (RT) The logic for detecting this defect is defined in Rule 3. A defect is identified when the calculation of staking rewards does not rely on any variable associated with the staking duration ($\neg StakeTime(var)$). In determining whether a variable is related to staking duration, SSR considers not only relevant variables in the DeFi staking model but also special time-related variables in Solidity smart contracts, such as *block.timestamp*.

$$isRT = Reward(re) \wedge \neg StakeTime(var) \wedge CalDepend(re, var) \quad (3)$$

(3) Single Liquidity Pool Reliance (SLR) The logic for detecting this defect is defined in Rule 4. A defect is identified when the staking rewards depend solely on a single liquidity pool. In Rule 4, $\{con \mid CalDepend(re, con), lpPool(con)\}$ represents the set of liquidity pool contracts on which the calculation of staking reward depends.

$$isSLR = \frac{Reward(re) \wedge \#Pool == 1 \wedge}{Pool = \{con \mid CalDepend(re, con), lpPool(con)\}} \quad (4)$$

(4) Omission in Status Update (OSU) This defect involves three scenarios: whether the staked amount and staking time are updated in the stake function; whether the staking reward and staking duration are updated in the *getReward* function; and whether the staked amount is updated in the *unStake* function. Taking the status update in the *unStake* function as an example, the detection logic is shown in Rule 5. If the staked tokens amount (*Amount(a)*) is not updated in the *unStake* function (*unStake(func)*), a defect is considered to exist.

$$isOSU = unStake(func) \wedge Amount(a) \wedge ModifyVar(func, a) \quad (5)$$

(5) Unsafe Verification (UV) This defect involves two scenarios: whether the staked amount is verified in the `unStake` function, and whether the staking reward is verified in the `getReward` function. Taking the `unStake` function as an example, the detection logic is shown as Rule 6. If the amount of staked tokens is not verified ($\neg \text{verifyVar}(\text{func}, a)$) before the token transfer is executed, a defect is considered to exist.

$$\text{isUV} = \text{unStake}(\text{func}) \wedge \text{Amount}(a) \wedge \neg \text{verifyVar}(\text{func}, a) \quad (6)$$

(6) Unauthorized Staking Asset Access (UAA) This defect involves two scenarios: transferring the user's staked token or staking rewards through a token transfer function, or directly modifying them. Taking staking rewards as an example, the detection logic for these two situations is in Rule 7 and Rule 8, respectively. Rule 7 states that if a function allows transferring staking rewards from the address `from` to another address, but does not validate whether the caller is `from` or check for the appropriate permissions, a defect is considered to exist. Rule 8 states that if a function can directly modify the staking rewards without permission checks, a defect is considered to exist.

$$\text{isUAA} = \text{RewardTrans}(\text{func}, \text{from}, \$_ \$) \wedge \neg \text{perCheck}(\text{func}, \text{from}) \quad (7)$$

$$\text{isUAA} = \text{Reward}(\text{re}) \wedge \text{modifyVar}(\text{func}, \text{re}) \wedge \neg \text{perCheck}(\text{func}, \text{from}) \quad (8) "$$